A Subexponential Time Algorithm for Elliptic Curve Discrete Logarithm

Akshima[2], Renas Bacho[2], Yu-Hsuan Huang[1], Eike Kiltz[1,2], Aysan Nishaburi[2], Samin Nooripoor[2], Noah Tittelbach[2], and Emiel Wiedijk[2]

[1] Max Planck Institute for Security and Privacy
[2] Ruhr University Bochum

Version 0.2, October 29, 2026

Abstract. Elliptic-curve groups are usually modeled as generic groups, in which computing discrete logarithms provably takes $2^{\Omega(b)}$ group operations for a group of b-bit prime order. OpenAI recently claimed an algorithm that solves the Boolean tensor contraction problem, that is, counts the satisfying assignments of a graph whose edges are bits and whose vertices are predicates, in time $2^{O(s/\sqrt{\log s})}$ for graphs with s edges and small maximum degree. Assuming this claim, we show that the discrete logarithm problem on elliptic curves of prime order over b-bit prime fields can be solved in expected time $2^{O(b/(\log b)^{1/4})}$, against $2^{b/2}$ for generic algorithms. Our algorithm is index calculus with randomly hashed factor bases, in which a contraction algorithm based on the Boolean tensor contraction problem finds the relations. Its only curve-specific ingredients are encodings of point validity and point addition by graphs with $O(b)$ edges, and it extends to curves with a fixed cofactor.
The result is purely asymptotic and does not imply improved concrete attacks on currently deployed elliptic curves, including NIST P-256, NIST P-521, and Ed25519.

1 Introduction

1.1 Motivation

Let $E : y^2 = x^3 + a_E x + b_E$ be an elliptic curve over a prime field $\mathbb{F}_p$, $p > 3$, whose group of points $\mathbb{G} = E(\mathbb{F}_p)$ has prime order r. We write $\mathbb{G}$ additively, with identity $\mathcal{O}$, and fix a generator P. For an integer u, $[u]A$ is the u-fold multiple of $A \in \mathbb{G}$. The elliptic curve discrete logarithm (ECDL) problem asks, given $Q \in \mathbb{G}$, for $\xi \in \mathbb{F}_r$ with $Q = [\xi]P$. We write $b = \lceil \log p \rceil$.[3]

Elliptic curves look generic. For general cryptographically relevant elliptic curves over prime fields, no subexponential-time algorithm for computing discrete logarithms is known. They are therefore usually modeled as *generic groups*: algorithms may only add group elements and compare them for equality, through an oracle. In that model, computing ξ from $Q = [\xi]P$ takes $\Omega(\sqrt{r}) = 2^{\Omega(b)}$ group operations [Nec94,Sho97], and Pollard's rho method [Pol78] matches this with about $\sqrt{\pi r/4} \approx 2^{b/2}$ operations. Accordingly, exponential hardness, "no $2^{o(b)}$ algorithm", is the standard assumption about the elliptic-curve discrete logarithm problem (ECDL).

A new algorithm for SAT. OpenAI recently announced [Ope26] an algorithm that counts the satisfying assignments of a k-CNF formula with n variables in time $2^{O(n \log\log n/\sqrt{\log n})}$. This is $2^{o(n)}$ and would refute the Exponential Time Hypothesis. The manuscript is under external review. Its engine is a more general result (Theorem D.1 there, recalled as Theorem 2): the Boolean tensor contraction problem on a graph with s edges, that is, counting the assignments of bits to the edges that satisfy a predicate at every vertex, can be solved in time $2^{O(s/\sqrt{\log s})}$, provided no vertex has more than about $s^{0.9}$ edges.

The question. Can this contraction algorithm be applied to ECDL?

The answer. Yes. Used as a black box, it solves ECDL in slightly subexponential time, $2^{O(b/(\log b)^{1/4})}$: this is $2^{o(b)}$, though not $2^{O(b^{1-\varepsilon})}$ for any $\varepsilon > 0$. There is no contradiction with the generic lower bound, because the algorithm computes on the coordinates of points, which a generic algorithm cannot do.

[3] All logarithms in this paper are base two.

1.2 Our Result

Our main result depends on an arbitrary tensor contraction solver. The claimed runtime follows from using the solver from [Ope26].

Theorem 1 (Informal version of Theorem 4). *Assume Theorem 2. There is a randomized algorithm that, given $Q \in \mathbb{G}$, computes $\xi \in \mathbb{F}_r$ satisfying $Q = [\xi]P$, never returns an incorrect value, and has expected running time*

$$2^{O\left(b/(\log b)^{1/4}\right)}.$$

Limitations. The result is asymptotic and conditional. The proof of Theorem 2 in [Ope26] yields a saving only for networks of size at least $2^{2^{17}}$,[4] so the algorithm says nothing about curves such as P-256 and Ed25519, for which Pollard's rho algorithm costs about 2^{128} group operations.

1.3 Technical Overview

The discrete logarithm problem asks us to recover ξ from P and $Q = [\xi]P$. Our algorithm follows the classical four-step index-calculus paradigm [Adl79,MvOV96]. Its main novelty is the use of a Boolean tensor contraction solver for relation finding.

- **Step 1 (Factor-base selection).** We choose m factor bases $F_1, \ldots, F_m \subseteq \mathbb{G}$, each of size $O(r^{1/m})$, using random hash functions. Unlike classical index calculus, the factor bases are not defined by a notion of smoothness, but by efficiently computable membership predicates.
- **Step 2 (Relation finding).** For random $u, v \in \mathbb{F}_r$, we seek relations of the form

$$[u]P + [v]Q = A_1 + \cdots + A_m, \qquad A_i \in F_i. \tag{1}$$

  A second-moment argument shows that suitable decompositions exist with constant probability over the factor-base selection and random target. The key technical contribution is to encode the decomposition problem (1) as a Boolean tensor network with $O(mb)$ edges and maximum degree $O(b^{3/4})$. Assuming the claimed tensor contraction algorithm, such relations can be found in time $2^{O(mb/\sqrt{\log b})}$ per attempt.
  A crucial ingredient of Step 2 is a Boolean Tensor Network encoding of integer multiplication with $O(b)$ edges and maximum degree $O(b^{3/4})$. The construction uses a two-stage Fourier-transform decomposition, allowing individual vertices to check polynomial-time computable operations on moderately large blocks of bits. This avoids the $O(b \log b)$ edges incurred by conventional fast multiplication circuits, while keeping the vertex degrees sufficiently small. These encodings form the basis for modular arithmetic, elliptic-curve addition, and factor-base membership tests.
- **Step 3 (Linear algebra).** We collect $N + 1$ relations, where $N = \sum_i |F_i| = O(mr^{1/m})$. Since there are more relations than factor-base elements, linear algebra over $\mathbb{F}_r$ yields a nontrivial linear combination that eliminates all factor-base points, giving

$$[U]P + [V]Q = \mathcal{O}.$$

- **Step 4 (Discrete logarithm extraction).** Writing $Q = [\xi]P$, the resulting equation implies $U + V\xi = 0 \pmod r$, and hence $\xi = -U/V$ whenever $V \neq 0$. Our randomization ensures this condition with probability $1 - 1/r$.

Balancing the cost of relation finding and linear algebra gives an expected running time of $2^{O(b/m + mb/\sqrt{\log b})}$. Choosing $m = \lceil (\log b)^{1/4} \rceil$ yields $2^{O(b/(\log b)^{1/4})}$.

Comparison with finite-field index calculus. In classical index-calculus algorithms for discrete logarithms over finite fields [Adl79,MvOV96], relations are obtained by finding elements whose integer or polynomial representations factor completely over a factor base of small primes or low-degree irreducible polynomials. Crucially, such relations can be generated and recognized using established techniques for integer or polynomial factorization and smoothness testing. For elliptic curves, there is no analogous notion of smoothness or efficiently computable factorization of group elements, which has been a central obstacle to

[4] For smaller sizes, an internal parameter $u_0 = \lfloor \frac{b}{2^{17}} \rfloor$ (defined in D.4.2 in [Ope26]) is zero. This means the tensor contraction algorithm does not recurse, instead falling back to brute force.

applying index calculus. Our approach circumvents this obstacle by defining factor bases through random hash functions on point coordinates and searching for additive decompositions of elliptic-curve points into elements of these bases. While factor-base membership is efficiently testable, finding such decompositions is the main computational challenge. Assuming the claimed Boolean tensor contraction algorithm, we can solve this decomposition problem in subexponential time. The remaining steps of relation collection and linear algebra follow the classical index-calculus paradigm.

Previous index-calculus approaches for elliptic curves. Index-calculus methods for elliptic curves have been investigated previously, notably through Semaev's summation polynomials [Sem04] and their development by Gaudry [Gau09] and Diem [Die11]. These approaches reduce relation finding to solving systems of polynomial equations, with subexponential algorithms known for certain families of extension-field curves. Amadori, Pintore, and Sala [APS18] investigated summation-polynomial methods specifically for prime-field elliptic curves, proposing a variant that avoids the traditional linear-algebra step by searching for a single factor-base relation. Our approach instead encodes additive point decompositions directly as compact, low-degree Boolean tensor networks, using the claimed contraction algorithm to find relations over prime fields.

Why use tensor contraction instead of SAT. A straightforward reduction of discrete logarithm over elliptic curves to SAT does not give the desired running-time bound. Although the unknown scalar ξ has only $O(b)$ bits, a standard circuit-to-CNF encoding also introduces variables for intermediate computations. Verifying $Q = [\xi]P$ by double-and-add in projective coordinates uses $O(b)$ group operations, each involving $O(1)$ field operations. Using $O(b \log b)$-size multiplication circuits, this gives a bounded-width CNF formula with $n = O(b^2 \log b)$ variables and $O(n)$ clauses. Even the stronger fixed-clause-density bound $2^{O(n/\sqrt{\log n})}$ of [Ope26] then gives only the upper bound $2^{O(b^2\sqrt{\log b})}$, which does not establish a subexponential algorithm in b.

Version history

- Version 0.1 (Oct 28, 2026): initial release
- Version 0.2 (Oct 29, 2026): Added technical overview and more related work to introduction; Definitions 1 and 2 rewritten for better clarity, expanded some proofs of Section 2; minor bug fixes

2 The Boolean Tensor Contraction Problem

In this section, we recall a result claimed in a recent document released by OpenAI [Ope26]. Our results are conditional on the correctness of this claim, which concerns an object called (Boolean) *tensor network*, defined as follows.

Definition 1 (Boolean Tensor Network). *A* Boolean tensor network *(BTN) is a pair* $\mathsf{BTN} = (\mathcal{G}, \{P_v\}_{v \in V})$*, consisting of*

- *an undirected multigraph* $\mathcal{G} = (V, E)$ *without self-loops, whose edges are listed in some fixed order;*
- *for every vertex* $v \in V$*, a Boolean predicate*

$$P_v : \{0,1\}^{\deg(v)} \to \{0,1\},$$

where $\deg(v)$ *is the number of edges (counting multiplicity) incident to* v*, listed in some fixed order.*

The degree of BTN *is defined as* $\max_{v \in V} \deg(v)$*. An* assignment *is a vector* $x = (x_e)_{e \in E} \in \{0,1\}^{|E|}$*, associating a bit* x_e *with every edge* $e \in E$*. An assignment* x *is called* satisfying *if*

$$P_v(x_{e_1}, \ldots, x_{e_{\deg(v)}}) = 1 \qquad \text{for all } v \in V,$$

where $e_1, \ldots, e_{\deg(v)}$ *are the edges incident to* v*, in their fixed order.*

The Boolean tensor contraction *of* BTN *is defined as*

$$\mathsf{Contract}(\mathsf{BTN}) := \sum_{x \in \{0,1\}^{|E|}} \prod_{v \in V} P_v(x_{e_1}, \ldots, x_{e_{\deg(v)}}).$$

Equivalently, Contract(BTN) *is the number of satisfying assignments. The* Boolean tensor contraction problem *asks to compute* Contract(BTN) *for a given Boolean tensor network* BTN*.*

The OpenAI document [Ope26] claims to solve this problem in subexponential time and polynomial space, when the maximum degree of the graph is sufficiently smaller than the total number of edges. The content of this paper relies on the correctness of this result, which we recall below, assuming that the graph $\mathcal{G}$ and each element of V and E can be described in $\mathrm{poly}(|E|)$ bits, and that each P_v can be evaluated in time $\mathrm{poly}(|E|)$.

Theorem 2 ((Simplified) Theorem D.1 of [Ope26]). *Let $\mathsf{BTN} = (\mathcal{G}, \{P_v\}_{v\in V})$ be a Boolean tensor network with $\mathcal{G} = (V, E)$ and $s = |E| \geq 2$. Assume that BTN has degree $O(s^{9/10})$, the number of vertices and the total description length of BTN are bounded by $\mathrm{poly}(s)$, and each predicate P_v can be evaluated in time $\mathrm{poly}(s)$, uniformly over all vertices $v \in V$. Then there is a deterministic algorithm computing $\mathsf{Contract}(\mathsf{BTN})$ exactly in time $2^{O(s/\sqrt{\log s})}$ and space $\mathrm{poly}(s)$.*

Remark 1. We explain how the simplified statement follows from Theorem D.1 of [Ope26]. First, the original theorem requires the graph to be bipartite. We achieve this by subdividing every edge with a new vertex whose predicate checks equality of its two incident bits. This doubles the number of edges, preserves the contraction, and does not increase the maximum degree beyond $\max\{2, \deg(\mathsf{BTN})\}$. Second, the original theorem computes contractions over the ring $\mathbb{Z}_{2^{P_0}}[X]/X^{\ell}$. We choose $\ell = 1$ and $P_0 = s + 1$, and interpret the Boolean values of each predicate as elements of $\mathbb{Z}_{2^{P_0}}$. Since $0 \leq \mathrm{Contract}(\mathsf{BTN}) \leq 2^s$, the contraction modulo 2^{s+1} uniquely determines its exact integer value.

The polynomial description-size and evaluation-time assumptions ensure that the hypotheses of Theorem D.1 are satisfied. Adjusting its size parameter by a constant factor accommodates the edge subdivision and the implicit constant in the degree bound. The claimed time and space bounds follow.

2.1 Language Encoding

Definition 2 (Encoding a Language). *Let $\mathcal{L} \subseteq \{0,1\}^m$ be a language and let $\mathsf{BTN} = (\mathcal{G}, \{P_v\}_{v\in V})$ be a Boolean tensor network with $\mathcal{G} = (V, E)$. We say that BTN* encodes *$\mathcal{L}$ if there are (explicitly provided) m distinct edges $e_1, \ldots, e_m \in E$, called the* interface edges*, such that for every $w = (w_1, \ldots, w_m) \in \{0,1\}^m$,*

$$w \in \mathcal{L} \iff \exists\, x \in \{0,1\}^{|E|} : \quad x \text{ is satisfying for } \mathsf{BTN} \quad \text{and} \quad x_{e_i} = w_i \text{ for all } i \in [m].$$

All edges outside the interface are called auxiliary edges*.*

The encoding is called parsimonious *if, for every $w \in \mathcal{L}$, there exists exactly one satisfying assignment x with $x_{e_i} = w_i$ for all $i \in [m]$. In particular, for a parsimonious encoding, $\mathsf{Contract}(\mathsf{BTN}) = |\mathcal{L}|$.*

We usually describe a language by a condition on words, for example $a \cdot b = z$ for the language $\mathcal{L} = \{(a, b, c) \mid a \cdot b = c\}$, where a, b, c are integers written in binary. Hence we naturally extend encodings to such relations.

The following lemma shows how to use Theorem 2 to find an element of a language that admits a compact, low-degree (BTN) encoding.

Lemma 1 (Finding a Word in the Language). *Assume Theorem 2. Let BTN be a Boolean Tensor Network with s edges and degree $O(s^{9/10})$. If a BTN encodes language $\mathcal{L}$, then an element of $\mathcal{L}$ can be found, or $\mathcal{L} = \emptyset$ detected, deterministically in time $2^{O(s/\sqrt{\log s})}$ and polynomial space. If the encoding is parsimonious, then $|\mathcal{L}|$ can also be computed deterministically within the same time and space bounds. Moreover, if $\mathcal{L} \neq \emptyset$, a uniformly random element of $\mathcal{L}$ can be sampled in expected time $2^{O(s/\sqrt{\log s})}$ and polynomial space.*

Proof. Let $\mathsf{BTN} = (\mathcal{G}, \{P_v\}_{v\in V})$ be a Boolean tensor network encoding $\mathcal{L}$, where $\mathcal{G} = (V, E)$ has $s = |E|$ edges and maximum degree $O(s^{9/10})$. Let $e_1, \ldots, e_m$ be its interface edges. In particular, $m \leq s$.

Fixing interface bits. We first show how to restrict the values of the interface edges without changing the graph. Fix an interface edge e_i, a bit $\beta \in \{0,1\}$, and choose an endpoint v of e_i. Suppose e_i is the j-th edge incident to v. Replace P_v by the predicate

$$P'_v(z_1, \ldots, z_{\deg(v)}) := P_v(z_1, \ldots, z_{\deg(v)}) \cdot \mathbf{1}[z_j = \beta].$$

All other predicates remain unchanged. A satisfying assignment of the modified BTN is therefore exactly a satisfying assignment of the original BTN with $x_{e_i} = \beta$.

This modification does not change the graph, the number of edges, or the maximum degree. Moreover, evaluating P'_v requires only one additional bit comparison, so the predicate remains polynomial-time computable. Several interface bits can be fixed by repeating this construction.

For any $0 \le i \le m$ and any $u = (u_1, \dots, u_i) \in \{0,1\}^i$, let $\mathsf{BTN}[u]$ denote the BTN obtained by fixing $x_{e_j} = u_j$ for all $j \in [i]$. Define

$$C(u) := \mathsf{Contract}(\mathsf{BTN}[u]).$$

Thus, $C(u)$ counts the satisfying assignments of the original BTN that agree with u on the first i interface edges. In particular, writing ε for the empty word, we have

$$C(\varepsilon) = \mathsf{Contract}(\mathsf{BTN}).$$

Furthermore, for every prefix u of length $i < m$, satisfying assignments extending u are partitioned according to the value of the next interface bit. Consequently,

$$C(u) = C(u0) + C(u1). \tag{2}$$

Finding a satisfying word. By the definition of an encoding, for every $u \in \{0,1\}^i$,

$$C(u) > 0 \quad \iff \quad \exists\, w \in \mathcal{L} \text{ whose first } i \text{ bits equal } u. \tag{3}$$

Indeed, if $C(u) > 0$, there exists a satisfying assignment whose first i interface bits equal u.

Since this means we can use a single tensor contraction to check whether a prefix can be extended into a word in the language we can construct a word in the language bit by bit. Given a prefix u with $C(u) > 0$ we can test both extensions $u0$ and $u1$ and continue with the one that satisfies $C(ub) > 0$. After m steps of this we have fixed all of the interface edges and u is a complete word in $\mathcal{L}$.

Counting the language. Now suppose that the encoding is parsimonious. By definition, every $w \in \mathcal{L}$ has exactly one satisfying extension to the auxiliary edges, while every $w \notin \mathcal{L}$ has none. Therefore, for every prefix $u \in \{0,1\}^i$,

$$C(u) = \left|\{w \in \mathcal{L} \mid (w_1, \dots, w_i) = u\}\right|. \tag{4}$$

Taking $u = \varepsilon$, we obtain

$$|\mathcal{L}| = C(\varepsilon) = \mathrm{Contract}(\mathsf{BTN}).$$

Thus a single application of Theorem 2 computes the cardinality of $\mathcal{L}$.

Uniform sampling. Assume that the encoding is parsimonious and $\mathcal{L} \neq \emptyset$. We construct a uniformly random element of $\mathcal{L}$ by choosing its bits sequentially. Suppose that the prefix u has already been chosen and $C(u) > 0$. Compute $C(u0)$ and set

$$N_0 := C(u0), \qquad N_1 := C(u) - N_0 = C(u1).$$

Choose the next bit according to

$$\Pr[w_{i+1} = 0 \mid u] = \frac{N_0}{C(u)}, \qquad \Pr[w_{i+1} = 1 \mid u] = \frac{N_1}{C(u)}. \tag{5}$$

These probabilities are well-defined because $C(u) > 0$, and the selected prefix again has positive contraction. They can be implemented exactly using standard techniques using $O(\log_2 C(u))$ random bits in expectation.

To prove uniformity, fix any $w = (w_1, \dots, w_m) \in \mathcal{L}$. By the chain rule and (5), the probability that the algorithm outputs w is

$$\begin{aligned}
\Pr[\text{output } w] &= \prod_{i=1}^{m} \Pr[w_i \mid w_1, \dots, w_{i-1}] \\
&= \prod_{i=1}^{m} \frac{C(w_1, \dots, w_i)}{C(w_1, \dots, w_{i-1})} \\
&= \frac{C(w)}{C(\varepsilon)} = \frac{1}{|\mathcal{L}|}.
\end{aligned}$$

The product telescopes, and the last equality follows from parsimony, which implies $C(w) = 1$ for every $w \in \mathcal{L}$. Thus every word in $\mathcal{L}$ is output with the same probability, and the sampling algorithm is exactly uniform.

Running time and space. Each network $\mathsf{BTN}[u]$ has exactly s edges and the same maximum degree as BTN. Its predicates remain polynomial-time computable. Consequently, every contraction $C(u)$ can be evaluated by Theorem 2 in time

$$T(s) = 2^{O(s/\sqrt{\log s})}$$

and polynomial space.

All three algorithms can be implemented using at most $m+1$ tensor contractions since in each iteration it suffices to compute the contraction $C(u0)$ and compute $C(u1) = C(u) - C(u0)$ where $C(u)$ is known from the previous iteration. Therefore, total runtime is

$$(s+1)\,T(s) + \mathrm{poly}(s) = 2^{O(s/\sqrt{\log s})},$$

The space requirement is dominated by the space needed for tensor contraction. □

2.2 Encodings for Simple Operations

We start with a few basic operations that admit simple linear-size and constant-degree BTN encodings.

Lemma 2. *On d-bit words, the following relations have parsimonious encodings with $O(d)$ edges and constant degree:*

- *addition $x + y = z$ of integers;*
- *comparison $x < y$;*
- *the zero test $x = 0$;*
- *the gated equality $\sigma \Rightarrow (x = y)$ for a selector bit σ;*
- *copying a word to f places, with $O(fd)$ edges.*

Note that we can replace any word with a public constant, by hardcoding the values of the edges.

Proof. We first give an explicit BTN for addition $a + b = z$ of two d-bit integers. Write $a = (a_0, \ldots, a_{d-1})$, $b = (b_0, \ldots, b_{d-1})$, and $z = (z_0, \ldots, z_d)$ in binary, with the least significant bit first.

The BTN contains vertices $v_0, \ldots, v_{d-1}$, one for each bit position. The interface edges carry the bits of a, b, and z (with one end terminating in a degree one vertex with tautology predicate). For $1 \le i < d$, an auxiliary edge between v_{i-1} and v_i contains the carry bit c_i, therefore we force $c_0 = 0$ and $c_d = z_d$.

For each $0 \le i < d$, the predicate P_{v_i} evaluates to 1 if and only if

$$a_i + b_i + c_i = z_i + 2c_{i+1}.$$

The network has $3d + 1$ interface edges and $d - 1$ auxiliary edges, hence $4d$ edges in total, and maximum degree at most 5.

We show that this BTN encodes the language $\mathcal{L} = \{(a, b, z) : a+b = z\}$. Fix an assignment $w = (a, b, z)$ to the interface edges. If w extends to a satisfying assignment, then all vertex predicates are satisfied, so

$$a_i + b_i + c_i = z_i + 2c_{i+1} \qquad (0 \le i < d).$$

Multiplying the i-th equation by 2^i and summing over all i, the carry terms telescope. Using $c_0 = 0$ and $c_d = z_d$, we obtain $a + b = z$. Thus $w \in \mathcal{L}$.

Conversely, suppose $a + b = z$. Starting with $c_0 = 0$, define the carry bits recursively by

$$c_{i+1} = \left\lfloor \frac{a_i + b_i + c_i}{2} \right\rfloor.$$

These carries yield the binary representation of $a + b$, including $c_d = z_d$, and satisfy all vertex predicates. Hence w extends to a satisfying assignment.

Moreover, the recursion uniquely determines every auxiliary carry bit from a and b. Therefore, each $w \in \mathcal{L}$ has exactly one satisfying extension, proving parsimony.

Comparison $a < b$ is implemented using a similar but simpler ripple chain, where the comparison state rather than a carry bit is propagated from the least significant to the most significant bit. The zero test $a = 0$ is implemented by an OR chain. Both constructions use $O(d)$ edges and constant-degree vertices, with uniquely determined intermediate bits.

For gated equality $\sigma \Rightarrow (a = b)$, introduce one vertex for each bit position i, checking $\sigma = 0$ or $a_i = b_i$. The selector σ is distributed to these vertices using a binary tree of equality predicates. Similarly, copying a d-bit word to f places is implemented by equality trees, one for each bit position, using $O(fd)$ edges. In both cases, all auxiliary bits are uniquely determined by the interface bits, so the encodings are parsimonious. □

2.3 Encoding for Integer Multiplication

In this subsection, we provide a linear-size, low-degree BTN that encodes integer multiplication.

The problem. Multiplying two d-bit numbers a and b has $2d$ input and $2d$ output bits, so any encoding needs $\Omega(d)$ edges. Two obvious encodings fail:

- As a circuit, even the best known fast circuits have about $d \log d$ wires, and each wire becomes an edge. This costs a factor $\log d$ in the size, more than the factor $\sqrt{\log d}$ that Theorem 2 saves.
- As a single vertex with a predicate checking $ab = z$, it has $4d$ ports, which breaks the degree cap of Theorem 2 when $d \approx s$.

The solution lies in a combination of the two, using the fast multiplication circuit for the Schönhage–Strassen algorithm [SS71] with four stages: transform, pointwise product, inverse transform, carries. Each stage maps $O(d)$ bits to $O(d)$ bits and is implemented by $O(\sqrt{d})$ vertices of degree $O(d^{3/4})$.

Decomposing an integer into base-2^k limbs, $a = \sum_{i<n} a_i 2^{ki}$ with $0 \le a_i < 2^k$, we view it as the polynomial $\hat{a}(X) = \sum_i a_i X^i$, which satisfies $\hat{a}(2^k) = a$. With $\hat{c} := \hat{a}\,\hat{b}$,

$$ab = \hat{c}(2^k) = \sum_{\ell \le 2n-2} c_\ell 2^{k\ell}, \qquad c_\ell := \sum_{i+j=\ell} a_i b_j. \tag{6}$$

The vector c is the *convolution* of the limb vectors. Since c_ℓ may exceed 2^k (it is below $n\,2^{2k}$), a final pass of *carries* turns (6) into the limbs of ab.

The discrete Fourier transform. A polynomial of degree below L is determined by its coefficients or by its values at L points. Multiplying coefficients is a dense convolution; multiplying values is pointwise. So we *evaluate* both factors at L points, multiply the values, and *interpolate*; if the product has degree below L, this returns its coefficients exactly. The *discrete Fourier transform* (DFT) evaluates at the powers of a root of unity ω. Then interpolation is again a DFT (with ω^{-1}), and the transform splits into smaller ones (Lemma 4). The DFT is commonly used over $\mathbb{C}$ with $\omega = e^{2\pi i/L}$, but predicates need exact values, so we work in a finite ring.

Definition 3 (DFT over a ring). *Let $\mathcal{A}$ be a commutative ring and $\omega \in \mathcal{A}$ with $\omega^L = 1$. Then ω is a* principal L-th root of unity *if $\sum_{i<L} \omega^{ji} = 0$ for all $0 < j < L$. For $x \in \mathcal{A}^L$, let $\mathrm{DFT}_\omega(x)_t := \sum_{j<L} x_j \omega^{jt} = x(\omega^t)$, where $x(X) = \sum_j x_j X^j$.*

The sum condition says that the powers of ω^j cancel, as evenly spaced points on a circle do in $\mathbb{C}$. In rings with zero divisors, such as $\mathbb{Z}/(2^J + 1)$, $\omega^j \ne 1$ does not imply it, so we require it directly.

Lemma 3 (Inversion and convolution). *Let ω be a principal L-th root of unity and L a unit in $\mathcal{A}$.*

1. *$x_j = L^{-1} \sum_{t<L} \mathrm{DFT}_\omega(x)_t\, \omega^{-jt}$; that is, the inverse transform is $L^{-1}\mathrm{DFT}_{\omega^{-1}}$.*
2. *If $x_j = y_j = 0$ for $j \ge n$, where $2n-1 \le L$, and $c_j = \sum_{i+l=j} x_i y_l$, then $\mathrm{DFT}_\omega(c)_t = \mathrm{DFT}_\omega(x)_t\, \mathrm{DFT}_\omega(y)_t$.*

Proof. Part 1. $\sum_t \mathrm{DFT}_\omega(x)_t\, \omega^{-jt} = \sum_i x_i \sum_t \omega^{(i-j)t}$. The inner sum is L for $i = j$ and 0 otherwise, since $\omega^L = 1$ lets us replace $i - j$ by its residue in $(0, L)$. Part 2. $x(\omega^t)\, y(\omega^t) = (xy)(\omega^t)$, and $xy = \sum_j c_j X^j$ has degree at most $2n - 2 < L$. □

The degree condition prevents *wraparound*: since $\omega^L = 1$, a coefficient at position $j \ge L$ would be added to position $j - L$. Padding with zeros to length $L \ge 2n - 1$ avoids it.

Splitting a transform into two stages. The DFT turns the convolution into L independent products, but it is itself dense, and a vertex checking a whole transform would see $\Theta(d)$ bits. The factorization behind the fast Fourier transform (FFT) splits it into two layers of small transforms. Intuitively, we arrange the inputs in an $L_1 \times L_2$ array, $x_{L_2 j_1 + j_2}$ in row j_1, column j_2. Stage 1 transforms each column (length L_1) and multiplies by the constants $\omega^{j_2 t_1}$ (*twiddle factors*); stage 2 transforms each row (length L_2). Columns are independent, and so are rows.

Lemma 4 (Four-step factorization). *Let $\omega^L = 1$ and $L = L_1 L_2$. For $x \in \mathcal{A}^L$, $j_2 < L_2$ and $t_1 < L_1$, set*

$$y_{j_2,t_1} := \omega^{j_2 t_1} \sum_{j_1 < L_1} (\omega^{L_2})^{j_1 t_1} \, x_{L_2 j_1 + j_2}. \tag{7}$$

Then for all $t_1 < L_1$ and $t_2 < L_2$,

$$\mathrm{DFT}_\omega(x)_{t_1 + L_1 t_2} = \sum_{j_2 < L_2} (\omega^{L_1})^{j_2 t_2} \, y_{j_2,t_1}. \tag{8}$$

Proof. Write $j = L_2 j_1 + j_2$ and $t = t_1 + L_1 t_2$. Then $jt = L_2 j_1 t_1 + j_2 t_1 + L_1 j_2 t_2 + L j_1 t_2$, and the last term vanishes in the exponent of ω. Insert into $\sum_j x_j \omega^{jt}$ and sum over j_1 first. □

Why only one split. The FFT splits recursively into $\log L$ stages, which minimizes operations. In a network, each stage boundary costs L words of edges, so $\log L$ stages would cost $\Theta(d \log d)$ edges, like a circuit. We split once, with $L_1, L_2 \approx \sqrt{L}$. Each small transform of size L_1 or L_2 can be performed in a single vertex. This is where the degree $O(d^{3/4})$ comes from: below, L and the word length are both $\Theta(\sqrt{d})$, so such a vertex sees $O(d^{1/4})$ words of $O(\sqrt{d})$ bits each.

The Low Degree BTN Encoding for Integer Multiplication. Having established the basic mechanisms by which we want to multiply integers, we are ready to construct the network for multiplication.

Theorem 3 (Integer multiplication network). *Let $d \geq 1$. The relation $ab = z$ between d-bit integers a, b and a $2d$-bit integer z has a parsimonious encoding by a BTN with fewer than $1100(d+1)$ edges and maximum degree below $136(d+1)^{3/4}$. Each vertex predicate can be evaluated in time polynomial in d.*

Choosing the parameters. We cut a and b into n limbs of k bits, pad the limb vectors with zeros to length L, apply DFTs of length L to them, and compute in the ring $\mathbb{Z}/(2^J + 1)$, whose elements take $J + 1$ bits. Three requirements tie these together:

- *No overflow:* a convolution coefficient is below $n 2^{2k}$ and must fit into the ring, so J must be somewhat larger than $2k$;
- *No wraparound:* $L \geq 2n \approx 2d/k$;
- *A root of unity:* we use $\omega = 2^{2J/L}$, which is only defined if L divides $2J$, so $L \leq 2J$.

Together, $d/k \lesssim L \lesssim J$, so $d \lesssim kJ$. With k and J proportional, this forces $k, J, L = \Theta(\sqrt{d})$. Then a vector of L ring elements has $L(J+1) = O(d)$ bits, which is what keeps the network linear in size. Concretely, we set:

- J: the least power of two with $J^2 \geq 64(d+1)$ (ring size);
- $k := J/4$ (limb width) and $n := \lceil d/k \rceil$ (number of limbs);
- $L = 2^\mu$: the least power of two with $L \geq 2n$ (transform length), split as $L = L_1 L_2$ with

$$L_1 := 2^{\lfloor \mu/2 \rfloor} \quad \text{and} \quad L_2 := 2^{\lceil \mu/2 \rceil};$$

- $\mathcal{A} := \mathbb{Z}/(2^J + 1)$ and $\omega := 2^{2J/L}$.

A ring element is stored in a $(J+1)$-bit word in *canonical form*, as its residue in $[0, 2^J]$. The following lemma checks that these choices meet the three requirements, and bounds the sizes we need for counting edges.

Lemma 5. *With the notation defined above:*

1. *$J \geq 16$ and $J^2 < 256(d+1)$.*
2. *$2n \leq L \leq J/2$; hence L divides $2J$ and $\max(L_1, L_2) \leq \sqrt{J}$.*
3. *ω is a principal L-th root of unity in $\mathcal{A}$, and L is a unit.*
4. *The convolution coefficients of d-bit a, b satisfy $c_\ell < 2^J$.*

Proof. Part 1. Since $d \geq 1$, $J^2 \geq 128$, and as J is a power of two, $J \geq 16$. If $J > 16$, then $J/2$ is a smaller power of two, so by minimality $(J/2)^2 < 64(d+1)$, that is, $J^2 < 256(d+1)$; for $J = 16$ this holds directly. *Part 2.* $L \geq 2n$ holds by definition. For the upper bound, $d < J^2/64$ and $k = J/4$ give $d/k < J/16$, so $n < J/16 + 1$. Since L is the least power of two with $L \geq 2n$, we have

$$L < 4n < J/4 + 4 \leq J/2,$$

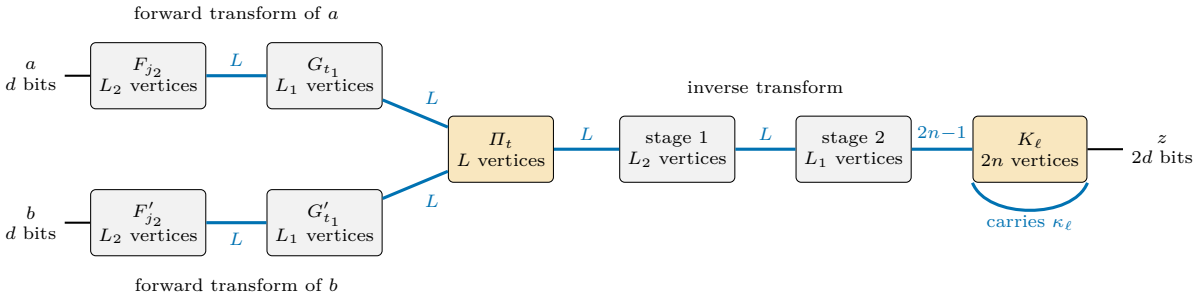


Fig. 1. The multiplication network. Boxes are layers of independent vertices; blue lines are vectors of words, labelled with their number. Transform vertices (grey) see at most $2\sqrt{J}$ words, product and carry vertices (orange) $O(1)$.

using $J \geq 16$. As L and J are powers of two with $L \leq J/2$, L divides $2J$, so $\omega = 2^{2J/L}$ is well defined. Finally,

$$L_1 \leq L_2 \leq \sqrt{2L} \leq \sqrt{J}.$$

Part 3. First, $\omega^{L/2} = 2^J = -1$ in $\mathcal{A}$, so $\omega^L = 1$. Now fix $0 < j < L$, write $j = 2^v j'$ with j' odd, and put $h := L/2^{v+1}$, an integer since $j < L$. Then $jh = j'L/2$, so $\omega^{jh} = (-1)^{j'} = -1$. Shifting the summation index i by h (modulo L) permutes the terms of $S = \sum_{i<L} \omega^{ji}$ and multiplies each by ω^{jh}, so $S = -S$. Since $2^J + 1$ is odd, 2 is a unit in $\mathcal{A}$, and $2S = 0$ gives $S = 0$. For the same reason $L = 2^\mu$ is a unit.

Part 4. Each c_ℓ is a sum of at most n products of two limbs below 2^k, so

$$c_\ell < n2^{2k} = n2^{J/2}.$$

As $n < J/16 + 1 \leq 2^{J/2}$ for $J \geq 16$, we get $c_\ell < 2^J$. □

The vertices. From now on, a and b also denote the limb vectors padded with zeros to length L, so

$$a = (a_0, \ldots, a_{n-1}, 0, \ldots, 0) \in \mathcal{A}^L,$$

and we write

$$\tilde{a} := \mathrm{DFT}_\omega(a), \qquad \tilde{b} := \mathrm{DFT}_\omega(b).$$

Except for the carries, every vertex checks "the output words are the canonical form of f(input words)" for an explicit f. Positions that are zero by construction (padding, and c_ℓ for $\ell \geq 2n-1$) are constants inside the predicates, not edges. The multiplication network is described as follows, see also Figure 1.

1. *Forward transforms* of a and of b, by Lemma 4. Stage 1 has a vertex F_{j_2} for each $j_2 < L_2$, with inputs the limbs $a_{L_2 j_1 + j_2}$ (interface edges; bits of a at positions $\geq d$ are fixed to 0) and outputs the words y_{j_2,t_1}; it checks (7). Stage 2 has a vertex G_{t_1} for each $t_1 < L_1$, with inputs y_{j_2,t_1} and outputs $\tilde{a}_{t_1 + L_1 t_2}$; it checks (8). The transform of b uses its own copies F'_{j_2} and G'_{t_1} of these vertices.
2. *Pointwise products:* for each $t < L$ a vertex Π_t checking $\tilde{c}_t = \tilde{a}_t \tilde{b}_t$.
3. *Inverse transform:* the same two stages applied to $\tilde{c}$ with ω^{-1} (also $(\omega^{-1})^L = 1$), the second stage also multiplying by L^{-1}. Outputs: $c_0, \ldots, c_{2n-2}$.
4. *Carries:* introduce auxiliary words κ_ℓ that contain the carry between limbs $\ell - 1$ and ℓ. There is no carry into the lowest limb or out of the highest one, so we fix $\kappa_0 = \kappa_{2n} = 0$; Lemma 6 shows that this is consistent. For each $\ell < 2n$ a vertex K_ℓ with inputs c_ℓ ($c_{2n-1} := 0$) and κ_ℓ, outputs the limb z_ℓ of z and $\kappa_{\ell+1}$, checking
$$c_\ell + \kappa_\ell = z_\ell + 2^k \kappa_{\ell+1}, \qquad 0 \leq z_\ell < 2^k, \tag{9}$$
with the bits of z at positions $\geq 2d$ fixed to 0.

A transform vertex does $O(L_2^2)$ operations modulo $2^J + 1$, the others $O(1)$, so all predicates take polynomial time.

Correctness and size. We now show correctness and upper bound the number of edges and the degree of the multiplication network.

Lemma 6 (Correctness). *For d-bit a, b and $2d$-bit z, there is a satisfying assignment with interface values (a, b, z) if and only if $z = ab$, and then exactly one.*

Proof. Fix a, b and follow the layers. A predicate "outputs are the canonical form of f(inputs)" forces its outputs. By Lemma 4, the forward layers force

$$\tilde{a} = \mathrm{DFT}_\omega(a) \quad \text{and} \quad \tilde{b} = \mathrm{DFT}_\omega(b).$$

By part 2 of Lemma 3 (with $2n \le L$), $\tilde{c} = \mathrm{DFT}_\omega(c)$. By part 1 of Lemma 3 and part 3 of Lemma 5, the inverse layer forces c modulo $2^J + 1$, which is c itself since $0 \le c_\ell < 2^J$ (part 4); in particular the outputs fixed to 0 are consistent.

In (9), z_ℓ and $\kappa_{\ell+1}$ are remainder and quotient of $c_\ell + \kappa_\ell$ by 2^k, so all carries and limbs are forced; by induction $\kappa_\ell < 2^J$, so they fit into words. Multiplying (9) by $2^{k\ell}$ and summing, the carries telescope:

$$\begin{aligned} ab &= \sum_{\ell<2n} c_\ell 2^{k\ell} \\ &= \sum_{\ell<2n} z_\ell 2^{k\ell} + \kappa_{2n} 2^{2nk}. \end{aligned}$$

As $0 \le \sum_\ell z_\ell 2^{k\ell} < 2^{2nk}$ and $ab < 2^{2d} \le 2^{2nk}$, the forced κ_{2n} is 0 and the z_ℓ are the digits of ab, which vanish from bit $2d$ on. So the constants are consistent, and the unique satisfying extension of (a, b) has $z = ab$. □

Canonical forms are what make the encoding parsimonious: a word can hold both u and $u + 2^J + 1$, and accepting both would give one product several satisfying assignments.

Lemma 7 (Edges and degree). *The network has fewer than $1100(d+1)$ edges and degree below $136(d+1)^{3/4}$.*

Proof. Edges. $4d$ interface edges, plus at most $8L$ words: $4L$ in the forward transforms, L products, L inside the inverse transform, and at most L each for the c_ℓ and the carries. With $L \le J/2$, $J \ge 16$ and $J^2 < 256(d+1)$,

$$\begin{aligned} 4d + 8L(J+1) &\le 4d + 4J(J+1) \\ &\le 4d + \tfrac{17}{4} J^2 \\ &< 1100(d+1). \end{aligned}$$

Degree. A transform vertex has at most $2\max(L_1, L_2)$ words, so at most

$$\begin{aligned} 2\sqrt{J}(J+1) &\le \frac{17}{8} J^{3/2} \\ &< \frac{17}{8} \cdot 64(d+1)^{3/4} \\ &= 136(d+1)^{3/4} \end{aligned}$$

ports. Product and carry vertices have fewer than $4(J+1)$ ports, which is less since $\sqrt{J} \ge 4$. □

Proof (of Theorem 3). By Lemma 6, the network encodes $ab = z$ parsimoniously, Lemma 7 bounds its edges and degree, and all predicates take polynomial time. □

2.4 Encoding for Modular Arithmetic

In this subsection, we provide a linear-size, low-degree BTN encoding for modular arithmetic.

Let $q \ge 2$ be a public modulus with $q < 2^d$. We store an element of $\mathbb{Z}_q$ in a d-bit word in *canonical form*, as its residue in $[0, q)$. As before, insisting on canonical forms keeps the encodings parsimonious.

A network does not need to reduce modulo q; it only needs to check a reduction. To check $z \equiv xy \pmod q$, we guess the quotient θ and check the integer identity $xy = \theta q + z$ together with $z < q$. Division with remainder makes θ unique, so guessing it keeps the encoding parsimonious.

Lemma 8. *On canonical words modulo q, the relations $x + y \equiv z \pmod q$ and $xy \equiv z \pmod q$ have parsimonious encodings with $O(d)$ edges and maximum degree $O(d^{3/4})$.*

Proof. Both networks check $x, y, z < q$ by Lemma 2. *Addition:* guess a bit β and a $(d+1)$-bit word s, and check $x + y = s$ and $z + \beta q = s$ by two additions of Lemma 2 on $(d+1)$-bit words. The i-th bit of the word βq is βq_i, so this word is β copied to the positions where q has a one, and 0 elsewhere. Since $x + y < 2q$, the only possible value is $\beta = [x + y \geq q]$. *Multiplication:* guess a d-bit θ and check $xy = \theta q + z$, using Theorem 3 for xy and for θq (with q hardcoded) and Lemma 2 for the addition. The quotient $\theta = \lfloor xy/q \rfloor < q$ fits into d bits, and every guessed word is determined by (x, y, z). □

Composing gadgets. To combine gadgets, we let them share words: wherever several gadgets use the same word, we connect their edges for that word by equality vertices (Lemma 2). A satisfying assignment of the combined network is then exactly a choice of values for all words that every gadget accepts. Each gadget adds $O(d)$ edges and keeps the degree at $O(d^{3/4})$. The combined network is parsimonious as long as the interface words determine all the other words, for example when they are intermediate results.

Corollary 1. *Let $\mathcal{L}$ be the set of $w \in \mathbb{Z}_q^{n_w}$ for which there is $u \in \mathbb{Z}_q^{n_u}$ such that (w, u) satisfies n_c constraints, each of the form $x + y \equiv z$, $xy \equiv z$ or $x \equiv c \pmod q$ for a public constant c, with w written as n_w canonical words. Then $\mathcal{L}$ has an encoding with $O((n_c + n_u + n_w)\, d)$ edges and maximum degree $O(d^{3/4})$. If u is unique for every $w \in \mathcal{L}$, the encoding is parsimonious.*

For example, (x, y) lies on $y^2 = x^3 + a_E x + b_E$ if and only if the constraints

$$x \cdot x \equiv u_1, \quad u_1 \cdot x \equiv u_2, \quad c_a \cdot x \equiv u_3, \quad u_2 + u_3 \equiv u_4, \quad u_4 + c_b \equiv u_5, \quad y \cdot y \equiv u_5$$

hold, with constants $c_a \equiv a_E$ and $c_b \equiv b_E$. Since the u_i are intermediate results, the encoding is parsimonious. Subtraction ($y + z \equiv x$) and division ($yz \equiv x$) come for free, and for prime q we can encode $x \not\equiv 0$ by guessing the inverse, as $xe \equiv 1$.

3 Elliptic Curve Discrete Logarithm

Let $E/\mathbb{F}_p$ be a nonsingular short-Weierstrass curve, with $p > 3$, given by

$$E : y^2 = x^3 + a_E x + b_E,$$

and suppose that $\#E(\mathbb{F}_p) = r$ is an odd prime. Write $\mathbb{G} = E(\mathbb{F}_p)$ as an additive group with identity $\mathcal{O}$, and let P be a generator of $\mathbb{G}$. The discrete logarithm problem is, given $Q \in \mathbb{G}$, to find $\xi \in \mathbb{F}_r$ such that $Q = [\xi]P$. We assume valid inputs and a known group order. The case $Q = \mathcal{O}$ has logarithm zero.

All logarithms are base two. Put

$$b = \lceil \log p \rceil.$$

By Hasse's bound [Gal12, Chapter 9],

$$|r - p - 1| \leq 2\sqrt{p}.$$

Hence $\log r = b + O(1)$. Throughout this section the curve has cofactor one, so checking that a point lies on the curve also establishes its membership in $\mathbb{G}$.

Theorem 4 (Discrete logarithms from tensor contraction). *Let $E/\mathbb{F}_p$, $p > 3$, have prime order r, let P generate $E(\mathbb{F}_p)$, and put $b = \lceil \log p \rceil$. For every integer $m \geq 2$ that is not too large compared with b, there is a randomized algorithm that, given $Q = [\xi]P$, returns ξ, never returns a wrong value, and uses, in expectation,*

- *$O(m^2 b\, r^{1/m})$ calls to a solver for the Boolean tensor contraction problem, each on a graph with $O(mb)$ edges and maximum degree $O(b^{3/4})$;*
- *$O(m^3\, r^{3/m}\, b^2)$ further bit operations for Gaussian elimination, and $\mathrm{poly}(b)$ bit operations per call for everything else;*
- *space $O(m^2\, r^{2/m}\, b)$ bits, plus the space of one call.*

If Theorem 2 holds, the expected running time is $2^{O(b/m + mb/\sqrt{\log b})}$, and for $m = \lceil (\log b)^{1/4} \rceil$ the expected running time and space are $2^{O(b/(\log b)^{1/4})}$.

Our proof of Theorem 4 follows the index-calculus paradigm, using a Boolean tensor contraction solver for relation finding. In Section 3.1, we construct compact, low-degree BTN encodings for elliptic-curve point addition. Section 3.2 introduces randomly hashed factor bases and proves that suitable decompositions exist with constant probability. In Section 3.3, we construct a BTN encoding the decomposition problem with $O(mb)$ edges and maximum degree $O(b^{3/4})$, allowing the tensor solver to find relations efficiently. Section 3.4 presents the full ECDL algorithm, including relation collection and discrete logarithm recovery via linear algebra. Finally, Section 3.5 analyzes the expected running time and space complexity and optimizes the parameter m to obtain the claimed bound.

Elliptic curves with fixed Cofactor. Theorem 4 assumes that the curve has *cofactor one*, that is, $\#E(\mathbb{F}_p) = r$ is prime. This removes all subgroup-membership checks. The general case of a fixed cofactor c requires only minor changes. Suppose $\#E(\mathbb{F}_p) = cr$, where $r > c$ is prime, and let $\mathbb{G}$ be the subgroup of order r. Since multiplication by c maps $E(\mathbb{F}_p)$ onto $\mathbb{G}$, each base point can be certified as $A = [c]S$ inside the network, which costs $O(\log c)$ additional point additions. During factor-base enumeration, we discard points outside $\mathbb{G}$ by checking $[r]A = \mathcal{O}$. The witness S need not be unique, since the decomposition encoding need not be parsimonious. Hence the same bound as in Theorem 4 holds for fixed c.

3.1 Encoding Elliptic Curve Point Addition

Checking point validity and point addition are the only curve-specific ingredients of the algorithm. We apply Corollary 1 with $q = p$ and $d = b$, which is possible since $p < 2^b$. We represent a point by a bit ι and coordinates (x, y): $\iota = 1$ and $x = y = 0$ for $\mathcal{O}$, and otherwise (x, y) is an affine point on E.

Corollary 2 (Point addition). *The relation $P_1 + P_2 = P_3$ on points of $E(\mathbb{F}_p)$ has a parsimonious encoding with $O(b)$ edges and maximum degree $O(b^{3/4})$. Consequently, $A_1 + \cdots + A_m = R$ has a parsimonious encoding with $O(mb)$ edges and the same maximum degree.*

Proof. By Corollary 1, it suffices to write the relation with $O(1)$ constraints whose guessed values are determined by the points. Validity of a point is $\iota \cdot \iota \equiv \iota$, $\iota x \equiv \iota y \equiv 0$, and the curve equation multiplied by $1 - \iota$.

Case distinction. The formula for P_3 depends on the case, but a network cannot branch. Instead, we compute one bit per case, called a *selector*, that is 1 exactly in that case, and use it to switch the constraints of that case on or off. This takes three steps.

1. *Zero tests.* We need bits $\zeta = [a \equiv 0]$. Guess ζ and e and check
$$a\zeta \equiv 0, \qquad ae \equiv 1 - \zeta, \qquad \zeta e \equiv 0.$$
If $a \not\equiv 0$, the first constraint forces $\zeta = 0$, and the second forces $e = a^{-1}$. If $a \equiv 0$, the second forces $\zeta = 1$, and the third forces $e = 0$. So ζ and e are determined. We use $\zeta_x = [x_1 - x_2 \equiv 0]$ and $\zeta_y = [y_1 + y_2 \equiv 0]$.
2. *Selectors.* The five cases and their selectors are

case	selector	constraints
$P_1 = \mathcal{O}$	$\sigma_1 = \iota_1$	$P_3 = P_2$
$P_2 = \mathcal{O} \neq P_1$	$\sigma_2 = (1 - \iota_1)\iota_2$	$P_3 = P_1$
$P_2 = -P_1$	$\sigma_3 = (1 - \iota_1)(1 - \iota_2)\zeta_x\zeta_y$	$P_3 = \mathcal{O}$
generic	$\sigma_4 = (1 - \iota_1)(1 - \iota_2)(1 - \zeta_x)$	$\iota_3 \equiv 0$, (10), (12)
doubling	$\sigma_5 = (1 - \iota_1)(1 - \iota_2)\zeta_x(1 - \zeta_y)$	$\iota_3 \equiv 0$, (11), (12)

Each selector is a product of bits, so it is a bit, and exactly one of them is 1.
3. *Switching.* Each constraint $C \equiv D$ in the row of σ is replaced by $\sigma(C - D) \equiv 0$. For $\sigma = 1$, this is $C \equiv D$. For $\sigma = 0$, it always holds. An equality of points such as $P_3 = P_2$ stands for $\iota_3 \equiv \iota_2$, $x_3 \equiv x_2$ and $y_3 \equiv y_2$.

Slope formulas. The generic and the doubling case guess the slope λ and check one of
$$\lambda(x_2 - x_1) \equiv y_2 - y_1 \qquad \text{(generic)}, \tag{10}$$
$$\lambda \cdot 2y_1 \equiv 3x_1^2 + a_E \qquad \text{(doubling)}, \tag{11}$$
together with the shared constraints
$$x_3 \equiv \lambda^2 - x_1 - x_2, \qquad y_3 \equiv \lambda(x_1 - x_3) - y_1. \tag{12}$$
In the generic case $x_2 - x_1$ is invertible. In the doubling case, two affine points on E with $x_1 = x_2$ have $y_2 = \pm y_1$, so $y_1 + y_2 \not\equiv 0$ forces $y_2 = y_1 \neq 0$, and $2y_1$ is invertible. Either way, λ is unique.

Parsimony. We show that all auxiliary variables are uniquely determined by the interface points P_1, P_2, P_3. First, the zero-test constraints uniquely determine the bits ζ_x, ζ_y and their corresponding inverse witnesses. Consequently, all five selectors $\sigma_1, \ldots, \sigma_5$ are uniquely determined. If $\sigma_4 = 1$ or $\sigma_5 = 1$, the corresponding slope equation uniquely determines λ, since its denominator is nonzero. In the remaining three cases, the additional constraint $(1 - \sigma_4 - \sigma_5)\lambda \equiv 0$ forces $\lambda = 0$.

To ensure uniqueness of the remaining auxiliary variables, we implement each switched constraint $\sigma(C - D) \equiv 0$ by first evaluating the expressions C and D unconditionally, using the parsimonious modular arithmetic encodings of Lemma 8, and then checking the gated equality. Thus, even when $\sigma = 0$, all intermediate values used to evaluate C and D remain uniquely determined.

Since all arithmetic intermediates and copied bits are uniquely determined by their inputs, every valid interface assignment satisfying $P_1 + P_2 = P_3$ has exactly one satisfying extension. Hence the encoding is parsimonious. □

3.2 Factor Bases and the Decomposition Problem

Fix an integer $m \geq 2$, the number of points in each decomposition. We will choose m at the end of the proof. The factor bases are sets $F_1, \ldots, F_m \subseteq \mathbb{G} \setminus \{\mathcal{O}\}$. Their discrete logarithms need not be known. We define them through membership tests that a tensor network can check without consulting a list.

Points as integers. For $A = (x, y) \in \mathbb{G} \setminus \{\mathcal{O}\}$, with canonical coordinates $0 \leq x, y < p$, put

$$\mathsf{enc}(A) = 2x + (y \bmod 2) \in \{0, \ldots, 2p - 1\}.$$

This encoding is injective: it determines x and the parity of y, while distinct possible ordinates for a fixed x are y and $p - y$, which have opposite parity.

Hashed factor bases. Fix a prime s with $2p \leq s \leq 4p$ and a threshold $1 \leq \tau \leq s$. A seed is a tuple

$$\mathcal{S} = (\alpha_i, \beta_i)_{i=1}^m \in (\mathbb{F}_s^* \times \mathbb{F}_s)^m.$$

It defines the hash functions

$$H_i(z) = (\alpha_i z + \beta_i) \bmod s$$

and the factor bases

$$F_i = \{A \in \mathbb{G} \setminus \{\mathcal{O}\} : H_i(\mathsf{enc}(A)) < \tau\}, \qquad i = 1, \ldots, m.$$

Hash outputs are represented by integers in $\{0, \ldots, s - 1\}$. The algorithm samples a uniform seed at the start of each trial and keeps it fixed for all targets of that trial. The threshold τ will be chosen in Lemma 13.

Definition 4 (Decomposition problem). *Given a seed $\mathcal{S}$ and a target $R \in \mathbb{G}$, find a tuple*

$$(A_1, \ldots, A_m) \in F_1 \times \cdots \times F_m \quad \textit{such that} \quad A_1 + \cdots + A_m = R,$$

or report that none exists.

For a fixed seed and target, Lemma 11 will encode this relation using $O(mb)$ edges. We first establish the properties of the factor bases used in the coverage and running-time analyses.

Lemma 9 (Factor-base properties). *Let the seed be uniform, and put $\rho = \tau/s$. For every i:*

1. $\Pr[A \in F_i] = \rho$ *for every* $A \in \mathbb{G} \setminus \{\mathcal{O}\}$*; hence*

$$\mathbb{E}[|F_i|] = (r - 1)\rho.$$

2. *For distinct* $A, B \in \mathbb{G} \setminus \{\mathcal{O}\}$,

$$\Pr[A, B \in F_i] = \frac{\tau(\tau - 1)}{s(s - 1)} \leq \rho^2.$$

3. $|F_i| \leq \tau$.

Moreover, $F_1, \ldots, F_m$ are independent.

Proof. For a fixed $z \in \mathbb{F}_s$, the value $H_i(z)$ is uniform because β_i is uniform. This gives part 1.

For distinct $z, z' \in \mathbb{F}_s$ and distinct $t, t' \in \mathbb{F}_s$, exactly one pair $(\alpha_i, \beta_i) \in \mathbb{F}_s^* \times \mathbb{F}_s$ maps z to t and z' to t', namely

$$\alpha_i = \frac{t - t'}{z - z'}, \qquad \beta_i = t - \alpha_i z.$$

Thus $(H_i(z), H_i(z'))$ is uniform over the $s(s-1)$ ordered pairs of distinct values. Exactly $\tau(\tau-1)$ of these pairs have both entries below τ, giving part 2.

Since $\mathsf{enc}(A) < 2p \le s$ and $\alpha_i \neq 0$, the map $A \mapsto H_i(\mathsf{enc}(A))$ is injective. This proves part 3. Independence follows from the independent choices of the hash parameters. □

The size bound controls the number of columns in the relation matrix. The membership probabilities will be used in Lemma 10 to show that a fixed target admits a decomposition with constant probability. Together, these bounds will determine the running time in Lemma 13.

Lemma 10 (Decomposition coverage). *Let $r \ge 100$ and $m \ge 2$, and put*

$$L = (r-1)\frac{\tau}{s}.$$

If

$$L \ge \max\{r^{1/m}, 10m\},$$

then, for every $R \in \mathbb{G}$,

$$\Pr_{\mathcal{S}}\big[R = A_1 + \cdots + A_m \text{ for some } A_i \in F_i\big] \ge \frac{2}{5},$$

where the seed $\mathcal{S}$ is uniform.

There are approximately $(r-1)^m/r$ tuples of nonidentity points summing to a fixed target R. Each survives the factor-base tests with probability $(\tau/s)^m$, giving an expected number of decompositions of approximately L^m/r. The proof bounds the second moment to turn this expectation into a constant probability of existence.

We now make this argument precise by bounding the first and second moments of the number of decompositions.

Proof. Put $n = r - 1$ and $\rho = \tau/s$, so $L = n\rho$. Fix $R \in \mathbb{G}$, and let X count its decompositions in $F_1 \times \cdots \times F_m$.

We first bound $\mu := \mathbb{E}[X]$. Choose the first $m-2$ nonidentity points arbitrarily. Of the n choices for the next point, at most one forces the final point to be $\mathcal{O}$. Thus at least $n^{m-2}(n-1)$ tuples sum to R. Each passes the membership tests with probability ρ^m, so

$$\mu \ge n^{m-2}(n-1)\rho^m = \left(1 - \frac{1}{n}\right)\frac{L^m}{n}.$$

We next bound $\mathbb{E}[X^2]$ by counting ordered pairs of decompositions. Identical pairs contribute μ. For distinct pairs, fix the first tuple and the set of $j < m$ positions where the tuples agree. There are at most

$$n^{m-j-1}$$

choices for the second tuple: choose all but one of its remaining entries, and the sum determines the last. By Lemma 9, the probability that both tuples pass the membership tests is at most ρ^{2m-j}.

Summing over the first tuple and the agreement positions, and using that μ is the number of first tuples times ρ^m, gives

$$\begin{aligned}\mathbb{E}[X^2] &\le \mu\left(1 + \sum_{j=0}^{m-1}\binom{m}{j} n^{m-j-1}\rho^{m-j}\right) \\ &\le \mu\left(1 + \frac{(L+1)^m}{n}\right).\end{aligned}$$

By Cauchy–Schwarz,

$$\begin{aligned}\Pr[X > 0] &\ge \frac{\mu^2}{\mathbb{E}[X^2]} \\ &\ge \frac{1 - 1/n}{n/L^m + (1 + 1/L)^m}.\end{aligned}$$

Since $L^m \ge r > n$, $L \ge 10m$, and $n \ge 99$, we conclude that

$$\Pr[X > 0] \ge \frac{98/99}{1 + e^{1/10}} > \frac{2}{5}.$$

3.3 Encoding the Decomposition Problem

Coverage ensures that suitable decompositions exist often enough. We next show that finding one is a small tensor-search problem.

Lemma 11 (Decomposition encoding). *For sufficiently large b, let*

$$2 \le m \le \left\lceil (\log b)^{1/4} \right\rceil .$$

Fix a seed and a target $R \in \mathbb{G}$. The relation

$$A_i \in F_i \quad (1 \le i \le m), \qquad A_1 + \cdots + A_m = R$$

has an encoding with $O(mb)$ edges and maximum degree $O(b^{3/4})$. Assuming Theorem 2, a decomposition can be found, or its nonexistence certified, deterministically in time

$$2^{O(mb/\sqrt{\log b})}$$

and space polynomial in mb. The implicit constants are independent of m, the seed, and the target.

Proof. By Corollary 2, the sum can be checked with $O(mb)$ edges, requiring each A_i to be a nonidentity curve point while allowing identity intermediate sums. Since $\mathbb{G}$ has prime order, no further subgroup check is needed.

For membership of $A_i = (x_i, y_i)$ in F_i, the bits of

$$\mathsf{enc}(A_i) = 2x_i + (y_i \bmod 2)$$

are copies of the coordinate bits. The encoding is less than $2p \le s$, so it is a canonical word modulo s, of length $O(b)$. Using Lemma 8, multiply it by α_i and add β_i modulo s. Then use Lemma 2 to check that the result is less than τ.

Each slot adds $O(b)$ edges. Including coordinate copies, the full network has $O(mb)$ edges and maximum degree $O(b^{3/4})$. Its satisfying assignments project exactly onto the desired decompositions; uniqueness of auxiliary values is not needed.

Since $b^{3/4} = O((mb)^{9/10})$, the network satisfies the degree bound of Lemma 1. Its description has size polynomial in mb, and its vertex predicates are computable in polynomial time by the cited arithmetic encodings. By Lemma 1, a witness can be found, or nonexistence certified, in time

$$2^{O(mb/\sqrt{\log(mb)})} = 2^{O(mb/\sqrt{\log b})}$$

and space polynomial in mb. Here $\log(mb) = \Theta(\log b)$ in the stated range. □

3.4 From Decompositions to Discrete Logarithms

We now describe the reduction. The factor bases remain fixed while we collect relations. Each relation involves points from the same fixed collection, which can therefore be eliminated by linear algebra. For a simplified version of the reduction see Figure 2.

Fix m and τ, sample a seed, and put

$$N = \sum_{i=1}^{m} |F_i|.$$

For the implementation below, we enumerate the factor-base elements to index the matrix columns. This enumeration is not used by the tensor membership tests, and its cost is included in Lemma 13.

Collecting relations. For each attempt, choose independent uniform scalars $u, v \in \mathbb{F}_r$, compute

$$R = [u]P + [v]Q,$$

and solve the decomposition problem using Lemma 11. For every fixed v, the uniform choice of u makes R uniform in $\mathbb{G}$. Fresh choices therefore give independent uniform targets.

If no decomposition exists, discard the attempt. Otherwise, record

$$[u]P + [v]Q = A_1 + \cdots + A_m, \qquad A_i \in F_i,$$

together with the chosen scalars u, v.

Discrete logarithms using a tensor solver

Input: $P, Q \in \mathbb{G}$, with P a generator.
Output: $\xi \in \mathbb{F}_r$ such that $Q = [\xi]P$.

1. **Choose the factor bases.** Sample a seed defining $F_1, \ldots, F_m$, and put $N = \sum_i |F_i|$. Construct a tensor network encoding the language

$$\mathcal{L} = \left\{ (R, A_1, \ldots, A_m) \in \mathbb{G} \times F_1 \times \cdots \times F_m \;\middle|\; R = \sum_{i=1}^{m} A_i \right\},$$

   where points are represented by their binary encodings.

2. **Find relations.** Choose independent uniform $u, v \in \mathbb{F}_r$ and set $R = [u]P + [v]Q$. Fix the target bits to R and use tensor contraction to find a member of $\mathcal{L}$ with this target. Each successful search gives a relation

$$[u]P + [v]Q = A_1 + \cdots + A_m.$$

   Store the relation and u, v. Repeat until $N + 1$ relations are collected; restart at Step 1 if $20(N + 1)$ attempts are insufficient.

3. **Cancel the factor-base points.** Find a nontrivial row dependency in the relation matrix over $\mathbb{F}_r$. Apply its coefficients to the stored relations to obtain

$$[U]P + [V]Q = \mathcal{O}.$$

4. **Recover the logarithm.** Return $\xi = -U/V$ if $V \neq 0$; otherwise restart at Step 1.

Fig. 2. Simplified discrete logarithm reduction; see Section 3.4 and Lemma 13 for details and parameter choices.

The relation matrix. The matrix has N columns, indexed by pairs (i, A) with $i \in \{1, \ldots, m\}$ and $A \in F_i$. Thus, a point belonging to several factor bases receives a separate column for each occurrence. For the j-th successful decomposition, define

$$M_{j,(i,A)} = \begin{cases} 1 & \text{if } A = A_{j,i}, \\ 0 & \text{otherwise.} \end{cases}$$

Each row contains exactly m ones, one for each factor base. Although its entries are only 0 and 1, we regard M as a matrix over $\mathbb{F}_r$.

Extracting the discrete logarithm. After collecting $N + 1$ successful relations, the matrix has more rows than columns. Compute a nontrivial linear combination of the rows that is zero in every column. Applying the same combination to the recorded group equations cancels every factor-base point and gives

$$[U]P + [V]Q = \mathcal{O},$$

where $U, V \in \mathbb{F}_r$ are the corresponding linear combinations of the stored scalars. Since $Q = [\xi]P$ and P has order r,

$$U + V\xi = 0 \pmod{r}.$$

If $V \neq 0$, return

$$\xi = -U/V \pmod{r}.$$

Trials and restarts. Each trial makes at most $20(N + 1)$ target attempts, stopping once $N + 1$ successful relations have been collected. If too few relations are obtained, or if $V = 0$, restart with a fresh seed.

The decomposition procedure receives only R and the seed, not the scalars u, v. Both the decomposition and the row dependency are chosen deterministically from their respective inputs. These choices ensure the following extraction guarantee.

Lemma 12 (Extraction). *Conditional on collecting $N + 1$ successful relations in a trial, the reduction in Section 3.4 recovers ξ with probability $1 - 1/r$.*

Proof. Fix the seed, and hence N. We may imagine sampling all $20(N+1)$ target pairs in advance, even if the trial stops early.

For each target $R = [u + \xi v]P$, fixing R leaves v uniform in $\mathbb{F}_r$, since every choice of v has exactly one matching value of u.

Condition on the full target sequence. This determines whether enough relations are collected, which relations are selected, and the chosen row dependency. The stored scalars v remain independent and uniform, including after conditioning on successful collection.

The coefficient V is a linear combination of these scalars with at least one nonzero coefficient, so it is uniform in $\mathbb{F}_r$. Thus $V \neq 0$ with probability $1 - 1/r$, in which case $\xi = -U/V$. □

3.5 Running Time

The coverage lemma concerns a fixed target and random factor bases. The reduction instead fixes the factor bases and uses many random targets. An averaging argument connects these two settings. We combine it with the size bounds to account for relation collection, restarts, and linear algebra.

Lemma 13 (Reduction cost). *For sufficiently large b and*

$$2 \leq m \leq \left\lceil (\log b)^{1/4} \right\rceil,$$

choose

$$\tau = \left\lceil \frac{s}{r-1} \max\{r^{1/m}, 10m\} \right\rceil.$$

Suppose each decomposition query can be answered deterministically in time

$$T = 2^{O(mb/\sqrt{\log b})}.$$

Then the reduction in Section 3.4 computes the discrete logarithm correctly in expected time

$$2^{O\left(b/m + mb/\sqrt{\log b}\right)}.$$

Proof. Factor-base size and construction. For sufficiently large b, the chosen threshold satisfies $\tau \leq s$. By Lemma 9,

$$N = \sum_{i=1}^{m} |F_i| \leq m\tau.$$

Hasse's bound gives $p/r = O(1)$, and hence $s/(r-1) = O(1)$. Therefore

$$m\tau = O(mr^{1/m} + m^2) \leq 2^{O(b/m + \log b)}.$$

The factor bases can be listed without scanning the group. For each slot i and each integer $0 \leq t < \tau$, compute

$$z = \alpha_i^{-1}(t - \beta_i) \bmod s.$$

If $z \geq 2p$, discard it. Otherwise, set $x = \lfloor z/2 \rfloor$ and solve

$$y^2 = x^3 + a_E x + b_E \pmod{p}.$$

Retain the root with parity $z \bmod 2$, if one exists. By injectivity of enc, this enumerates every point of F_i exactly once.

Quadratic-residuosity testing and square-root extraction take expected polynomial time in b; see [MvOV96, Algorithm 3.34 and Fact 3.35]. The case of a zero right-hand side is handled directly. Thus the list is exact, and the total expected construction time is

$$m\tau \operatorname{poly}(b).$$

Indexing the resulting columns, for example by sorting their encodings, is absorbed in this bound.

The prime s can be selected once in expected polynomial time. Indeed, the prime number theorem [MvOV96, Fact 2.95] implies that a uniform integer in $[2p, 4p]$ is prime with probability $\Theta(1/b)$ for sufficiently large p. Each candidate can be tested deterministically in polynomial time [AKS04].

Collecting relations. For fixed factor bases, let f be the fraction of targets in $\mathbb{G}$ admitting a decomposition. The choice of τ satisfies the hypotheses of Lemma 10, so averaging over the random factor bases gives $\mathbb{E}[f] \geq 2/5$. Call the factor bases good if $f \geq 1/5$. Since $0 \leq f \leq 1$,

$$\frac{2}{5} \leq \mathbb{E}[f] \leq \frac{1}{5} + \frac{4}{5}\Pr[f \geq 1/5].$$

Hence the factor bases are good with probability at least $1/4$.

For good factor bases, each independent uniform target succeeds with probability at least $1/5$. Collecting $N+1$ relations therefore takes at most $5(N+1)$ attempts in expectation. By Markov's inequality, at least $N+1$ relations are obtained within $20(N+1)$ attempts with probability at least $3/4$.

By Lemma 12, the probability that a trial succeeds is therefore at least

$$\frac{1}{4} \cdot \frac{3}{4} \cdot \left(1 - \frac{1}{r}\right) > \frac{1}{6}$$

for sufficiently large r. Thus a constant number of trials suffices in expectation.

Total cost. Each trial makes $O(N+1)$ decomposition queries, uses polynomial time per target, and performs Gaussian elimination in $O(N^3)$ field operations when enough relations have been collected. Including construction, its expected cost is at most

$$\begin{aligned} &m\tau\,\mathrm{poly}(b) \\ &\quad + O(N+1)\big(T + \mathrm{poly}(b)\big) \\ &\quad + O((N+1)^3)\,\mathrm{poly}(b). \end{aligned}$$

Substituting the bounds on $m\tau$, N, and T gives

$$2^{O\left(b/m + mb/\sqrt{\log b}\right)}.$$

The polynomial factors are absorbed into this bound in the stated range of m.

Fix the selected prime s. Let C be the cost of one trial and p_{succ} its success probability. Fresh trials are independent, so the expected total trial cost is $\mathbb{E}[C]/p_{\mathrm{succ}}$, even though a trial's cost may depend on whether it succeeds. Since $p_{\mathrm{succ}} > 1/6$, this preserves the bound. The one-time cost of selecting s is polynomial in expectation and is absorbed as well. Every returned value is correct by the extraction equation in Section 3.4. □

We can now choose the number of factors to balance the number of relations against the cost of each decomposition query.

Proof (of Theorem 4). For sufficiently large b, choose

$$m = \left\lceil (\log b)^{1/4} \right\rceil$$

and choose τ as in Lemma 13. By Lemma 11, decomposition queries have the required running time. Applying Lemma 13 gives expected time

$$2^{O\left(b/m + mb/\sqrt{\log b}\right)} = 2^{O\left(b/(\log b)^{1/4}\right)}.$$

The algorithm never returns an incorrect value. The finitely many smaller parameter sizes can be handled by exhaustive search. □

Remark 2 (Cost with an arbitrary contraction solver). Our reduction can use any deterministic Boolean tensor contraction solver. We first state its complexity independently of the solver's running time.

For m and τ as in Lemma 13, the expected number of solver calls is

$$O(m^2 b r^{1/m}),$$

each on a BTN with $O(mb)$ edges and maximum degree $O(b^{3/4})$. The expected cost of Gaussian elimination is $O(m^3 r^{3/m} b^2)$ bit operations, and the reduction uses $O(m^2 r^{2/m} b)$ bits of space, excluding the solver's workspace. All remaining operations require polynomial time per solver call.

More generally, suppose that for some constant $\alpha > 0$, Boolean tensor contraction on a BTN with ℓ edges and degree $O(\ell^{9/10})$ can be solved in time

$$2^{O(\ell/(\log \ell)^{\alpha})}$$

and polynomial space. The proofs of Lemmas 11 and 13 extend to any polylogarithmic m, since they only require $\log(mb) = \Theta(\log b)$ and $r^{1/m} \geq 10m$, which hold for sufficiently large b. Consequently, our ECDL algorithm has expected running time

$$2^{O(b/m + mb/(\log b)^{\alpha})}.$$

Choosing $m = \lceil (\log b)^{\alpha/2} \rceil$ yields

$$2^{O(b/(\log b)^{\alpha/2})}.$$

AI Disclosure

We used large language models (Claude and ChatGPT) in preparing this paper. They assisted with developing and checking the algorithm as well as drafting parts of the text and proofs, reviewing the manuscript for correctness, consistency and presentation, and LaTeX typesetting. The authors checked all statements and proofs and take full responsibility for the content.

Our main result is conditional on Theorem D.1 of [Ope26], which has not been peer-reviewed; the authors did not attempt to independently verify that theorem.

References

Adl79. Leonard M. Adleman. A subexponential algorithm for the discrete logarithm problem with applications to cryptography. In *20th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 55–60, 1979. 2

AKS04. Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004. 17

APS18. Alessandro Amadori, Federico Pintore, and Massimiliano Sala. On the discrete logarithm problem for prime-field elliptic curves. *Finite Fields and Their Applications*, 51:168–182, 2018. 3

Die11. Claus Diem. On the discrete logarithm problem in elliptic curves. *Compositio Mathematica*, 147(1):75–104, 2011. 3

Gal12. Steven D. Galbraith. *Mathematics of Public Key Cryptography*. Cambridge University Press, 2012. 11

Gau09. Pierrick Gaudry. Index calculus for abelian varieties of small dimension and the elliptic curve discrete logarithm problem. *Journal of Symbolic Computation*, 44(12):1690–1702, 2009. 3

MvOV96. Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996. 2, 17

Nec94. V. I. Nechaev. Complexity of a determinate algorithm for the discrete logarithm. *Mathematical Notes*, 55(2):165–172, 1994. 1

Ope26. OpenAI. The exponential time hypothesis is false, 2026. Preprint, 17 October 2026, revised 25 October 2026. Theorem D.1, p. 53. 1, 2, 3, 4, 19

Pol78. John M. Pollard. Monte Carlo methods for index computation (mod p). *Mathematics of Computation*, 32(143):918–924, 1978. 1

Sem04. Igor A. Semaev. Summation polynomials and the discrete logarithm problem on elliptic curves. Cryptology ePrint Archive, Report 2004/031, 2004. 3

Sho97. Victor Shoup. Lower bounds for discrete logarithms and related problems. In *Advances in Cryptology – EUROCRYPT '97*, volume 1233 of *Lecture Notes in Computer Science*, pages 256–266. Springer, 1997. 1

SS71. Arnold Schönhage and Volker Strassen. Schnelle Multiplikation großer Zahlen. *Computing*, 7:281–292, 1971. 7